Fourier Neural Operator: Learning Solution Operators in Spectral Space

Krishna Kumar

University of Texas at Austin

krishnak@utexas.edu

Outline

1. From CNNs to Kernel Operators: The Conceptual Leap
2. Fourier Transform: The Mathematical Foundation
3. Fourier Neural Operator Architecture
4. Example 1: 1D Burgers Equation
5. Example 2: 2D Darcy Flow

Learning Objectives

- Understand the connection between CNNs and kernel operators
- Master Fourier Transform fundamentals for neural operators
- Learn the FNO architecture: spectral convolution layers
- Implement FNO for 1D Burgers equation
- Apply FNO to 2D Darcy Flow problem
- Explore mesh independence and super-resolution capabilities

▸ Open Notebook: FNO

A temperature field becomes smoother

Consider heat flow on a periodic ring. The input and output are entire temperature fields.

$$u_0(x) = \sin x + \frac{1}{2}\sin 3x + \frac{1}{4}\sin 7x, \qquad u_0 \longmapsto u(\cdot, T).$$

The special inputs

Each sine keeps its spatial shape under diffusion. Its amplitude changes by a factor that depends on its frequency.

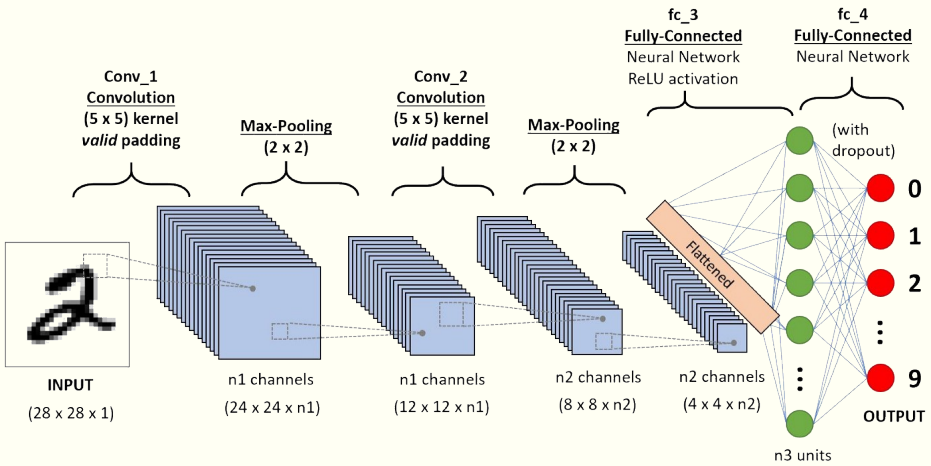
- A spectral solver derives those factors from the PDE.
- A Fourier layer learns factors from input-output fields.

▸ Heat-flow intuition notebook

Local convolution in a CNN

A shared stencil combines neighboring input values. For a one-dimensional periodic blur,

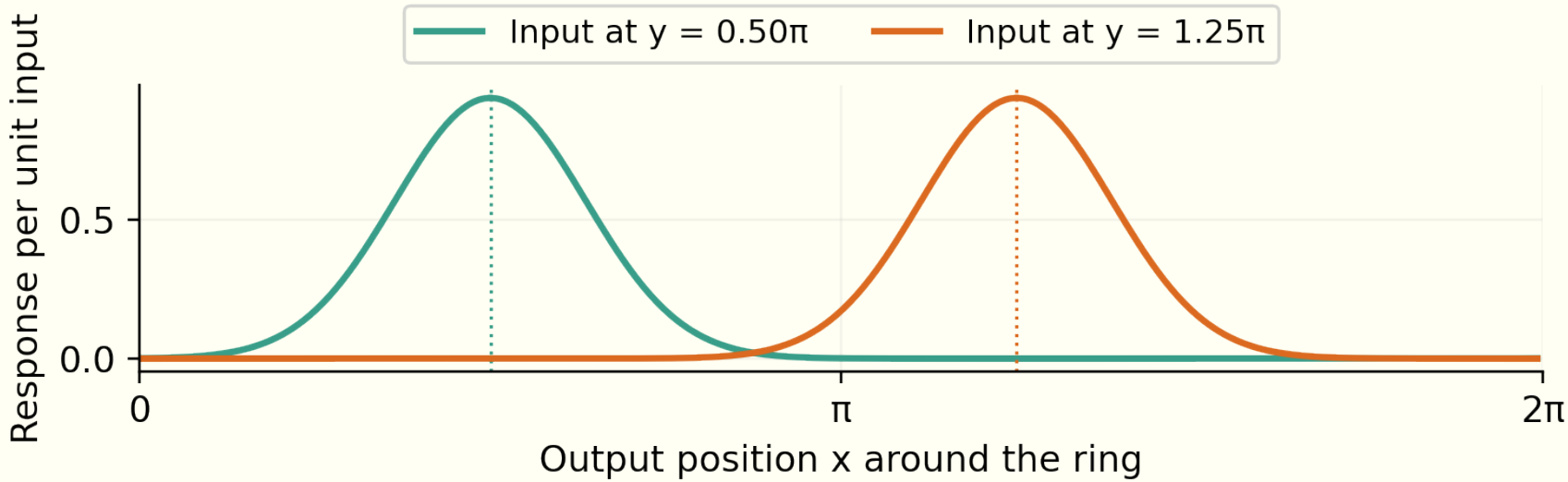
$$y_j = \frac{1}{3}(u_{j-1} + u_j + u_{j+1}).$$



- Shifting the input shifts the output of the convolution.
- A short stencil gives a local interaction in one layer.

The influence of one initial heat input

Place a unit amount of initial heat at position y. The resulting temperature at x and time T is $G_T(x, y)$.



Green's function

$G_T(x, y)$ gives the response at output position x to a localized input at y.

For the periodic ring, shifting the input shifts the response.

A kernel collects the point contributions

An interval near y contributes $u_0(y)\,dy$ units of initial heat. Its contribution at x after time T is $G_T(x, y)u_0(y)\,dy$.

$$\boxed{u(x, T) = \int_0^{2\pi} G_T(x, y)u_0(y)\,dy.}$$

The same structure defines an integral kernel operator,

$$(\mathcal{K}v)(x) = \int_\Omega \kappa(x, y)v(y)\,dy.$$

- y locates an input contribution and x locates the output.
- The kernel quantifies influence between the two locations.
- Physics may specify the kernel, or a model may learn it.

Translation invariance gives convolution

On a periodic ring with constant diffusivity, influence depends only on separation.

$$G_T(x, y) = g_T(x - y).$$

The integral operator becomes circular convolution,

$$\boxed{(\mathcal{K}_g v)(x) = \int_0^{2\pi} g(x - y)v(y)\, dy.}$$

The condition matters

A general Green's function depends on both x and y. Boundaries and variable coefficients can prevent this convolution form.

The Fourier representation below applies to the translation-invariant case.

Outline

1. From CNNs to Kernel Operators: The Conceptual Leap
2. Fourier Transform: The Mathematical Foundation
3. Fourier Neural Operator Architecture
4. Example 1: 1D Burgers Equation
5. Example 2: 2D Darcy Flow

Fourier coordinates on the ring

Use the periodic interval $[0, 2\pi)$ and integer wave numbers k.

$$\widehat{v}_k = \frac{1}{2\pi} \int_0^{2\pi} v(x) e^{-ikx}\, dx, \qquad v(x) = \sum_{k \in \mathbb{Z}} \widehat{v}_k e^{ikx}.$$

On a uniform grid $x_j = 2\pi j / N$, the normalized FFT computes

$$\widehat{v}_k = \frac{1}{N} \sum_{j=0}^{N-1} v(x_j) e^{-ikx_j}.$$

- The coefficients describe amplitudes and phases of waves.
- The inverse transform adds the waves to reconstruct values.
- The FFT changes coordinates without evolving the field.

Operations in Fourier coordinates

Differentiation

$$\widehat{\partial_x v}_k = ik\widehat{v}_k, \qquad \widehat{\partial_{xx} v}_k = -k^2\widehat{v}_k.$$

Convolution with the stated normalization

$$\widehat{\mathcal{K}_g v}_k = R(k)\widehat{v}_k, \qquad R(k) = \int_0^{2\pi} g(s)e^{-iks}\, ds.$$

Squared norm

$$\frac{1}{2\pi}\int_0^{2\pi} |v(x)|^2\, dx = \sum_k |\widehat{v}_k|^2.$$

A Fourier wave keeps its shape under convolution

Apply the kernel to one input wave $v(y) = e^{iky}$.

$$
\begin{aligned}
(\mathcal{K}_g e^{ik\cdot})(x) &= \int_0^{2\pi} g(x-y)e^{iky}\,dy \\
&= e^{ikx}\int_0^{2\pi} g(s)e^{-iks}\,ds \\
&= R(k)e^{ikx}.
\end{aligned}
$$

The substitution is $s = x - y$, with separation interpreted periodically.

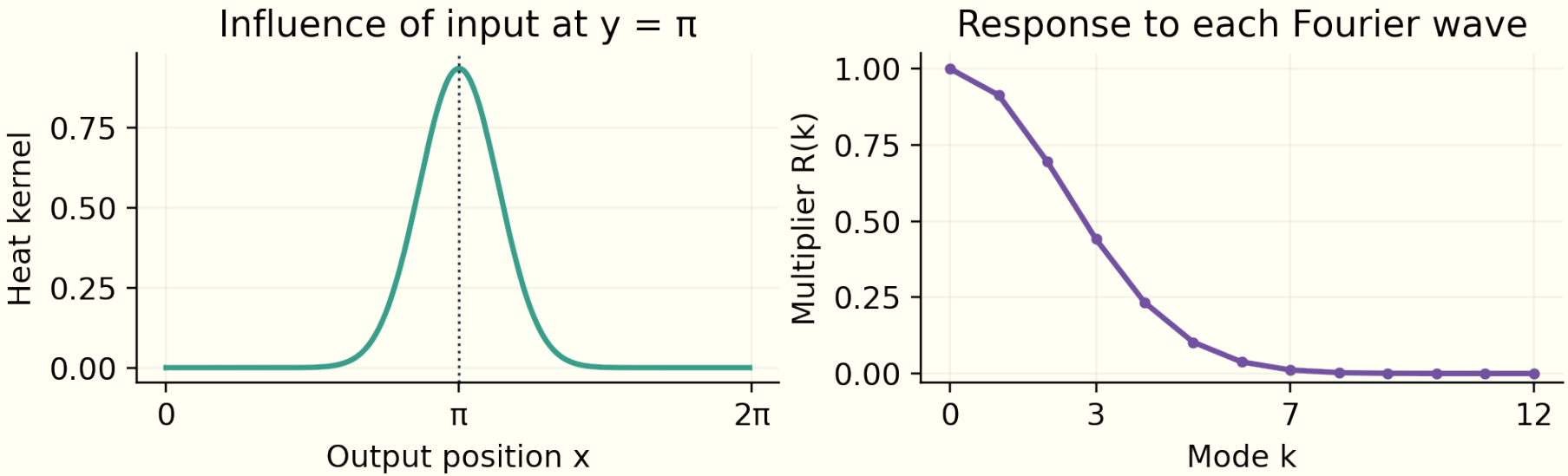
The eigenfunction connection

The frequency stays fixed. The complex number $R(k)$ changes amplitude and phase.

The heat kernel and its Fourier multipliers

For $u_t = \nu u_{xx}$, use $\nu = 0.14$ and $T = 0.65$. Mode k satisfies

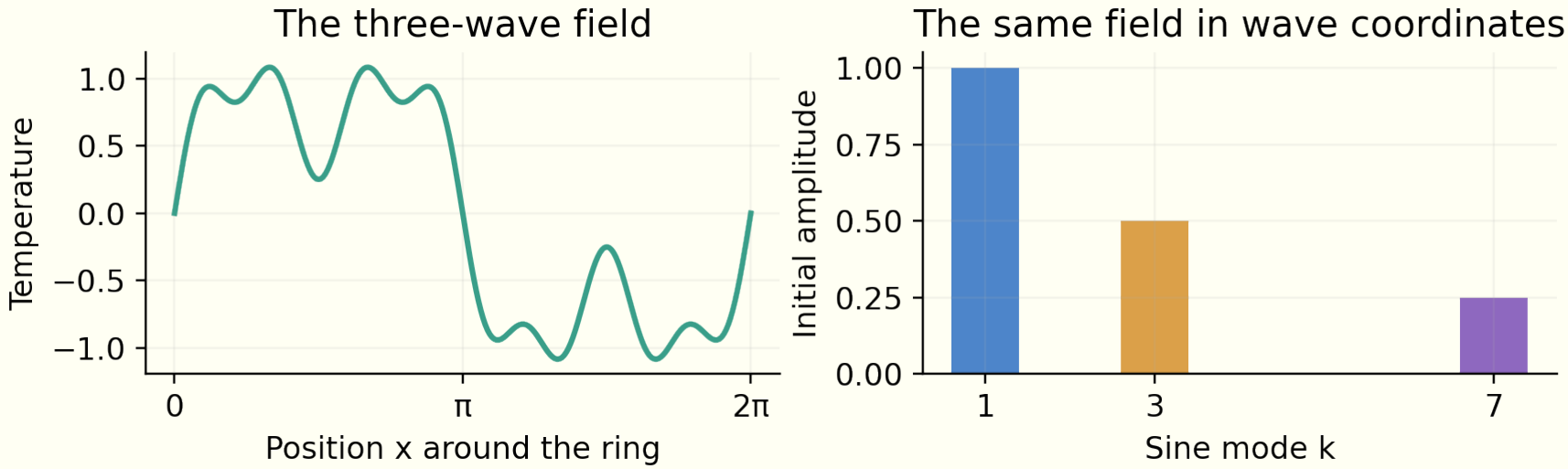
$$\frac{d\widehat{u}_k}{dt} = -\nu k^2 \widehat{u}_k, \qquad R_T(k) = e^{-\nu k^2 T}.$$



The point-influence kernel and the modal factors describe the same solution operator.

$$\mathcal{L}_T u_0 = g_T * u_0 = \mathcal{F}^{-1}[R_T \mathcal{F}(u_0)].$$

Wave coordinates and spectral evolution



For the three-wave heat example,

$$\begin{bmatrix} q_1(T) \\ q_3(T) \\ q_7(T) \end{bmatrix} = \mathrm{diag}(0.913, 0.441, 0.0116) \begin{bmatrix} 1 \\ 0.5 \\ 0.25 \end{bmatrix}.$$

The multipliers scale the initial amplitudes. The final amplitude of mode 3 is $0.5 \times 0.441 \approx 0.220$.

Outline

1. From CNNs to Kernel Operators: The Conceptual Leap
2. Fourier Transform: The Mathematical Foundation
3. Fourier Neural Operator Architecture
4. Example 1: 1D Burgers Equation
5. Example 2: 2D Darcy Flow

Learning the global convolution

Supply initial-final field pairs and fit the response $R_\theta(k)$. The coordinate transforms remain fixed.

$$\mathcal{K}_\theta v = \mathcal{F}^{-1}[R_\theta(k)\widehat{v}_k].$$

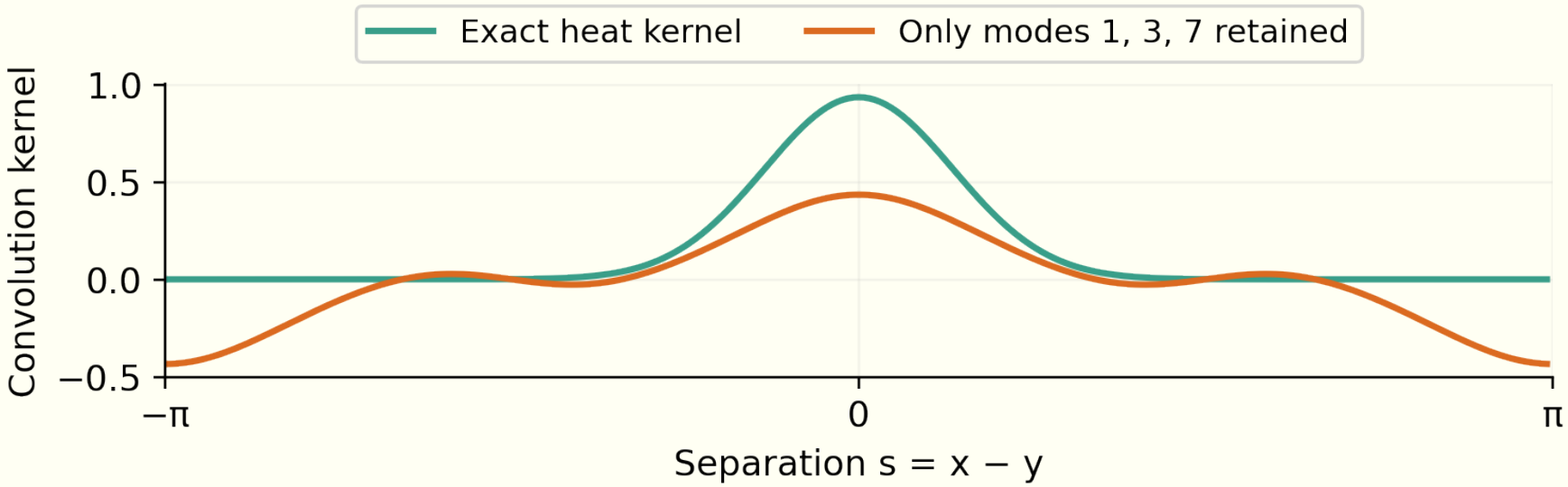
The equivalent periodic spatial kernel is

$$\boxed{g_\theta(s) = \frac{1}{2\pi}\sum_{k\in\mathcal{M}} R_\theta(k)e^{iks}.}$$

- The retained set $\mathcal{M}$ specifies the spectral branch.
- Set unretained multipliers to zero for this truncated layer.
- Each retained Fourier wave extends across the domain.

Li et al., Fourier Neural Operator, Section 4.

Three learned modes specify a restricted operator



The two kernels give the same heat output on the represented span

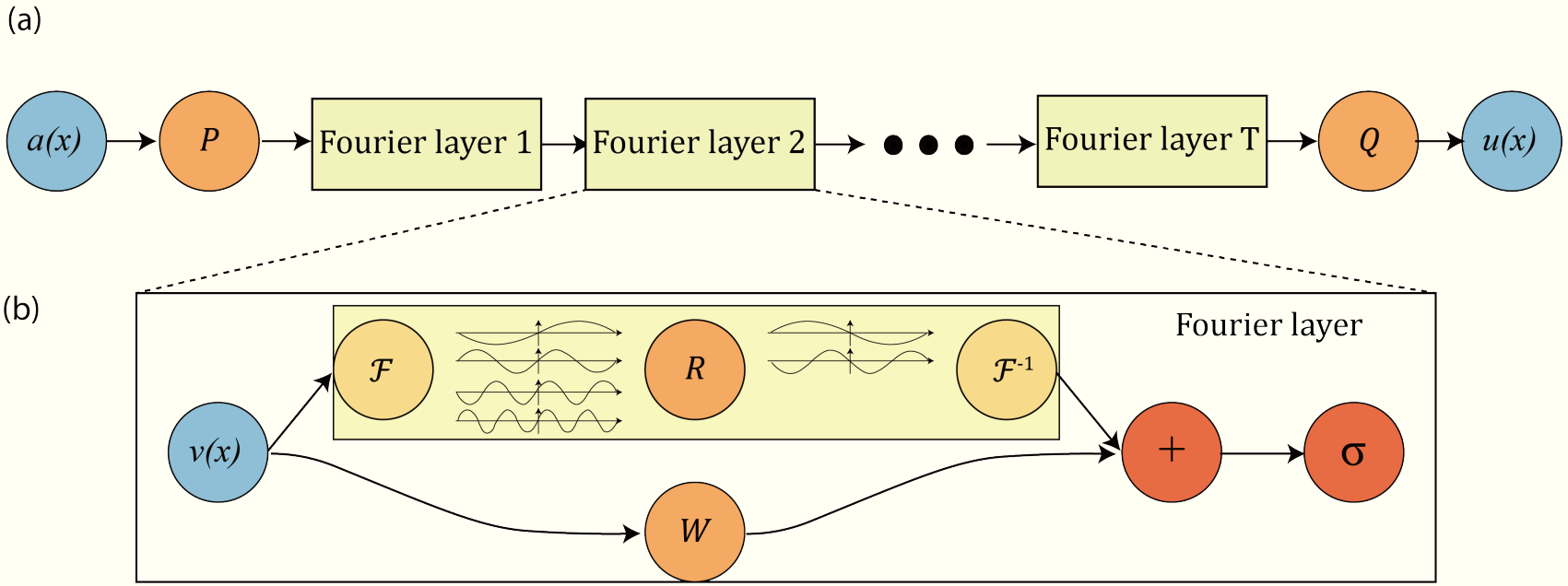
$$\operatorname{span}\{\sin x, \sin 3x, \sin 7x\}.$$

Input $v = 1$ gives outputs 1 and 0, respectively.

What the data identify

Three fitted responses determine the action on those modes. The truncation convention determines the extension to unretained modes.

Complete FNO Architecture



Fourier Neural Operator architecture.

Architecture consists of:

- **Lifting:** $P(u) \to v_0$ (embed input to high-dimensional space)
- **Fourier Layers:** $v_{\ell+1} = \sigma(W_\ell v_\ell + \mathcal{K}_\ell v_\ell)$ (4 layers)
- **Projection:** $Q(v_4) \to$ output (map back to output space)

The spectral branch acts globally

At each retained frequency, transform the channel coefficients,

$$\widehat{z}_o(k) = \sum_{i=1}^{C_{\text{in}}} R_{oi}(k)\widehat{v}_i(k), \qquad k \in \mathcal{M}.$$

Set the spectral output to zero outside $\mathcal{M}$, then reconstruct z.

$$z(x) = \sum_{k\in\mathcal{M}} \widehat{z}(k)e^{ikx}.$$

- $R(k)$ mixes channels at a fixed frequency.
- Its inverse Fourier representation is a matrix-valued global kernel.
- The mode cutoff is an architectural choice, not a guarantee of accuracy.

Global and local transformations in one FNO block

The spectral and pointwise paths process the same input features.

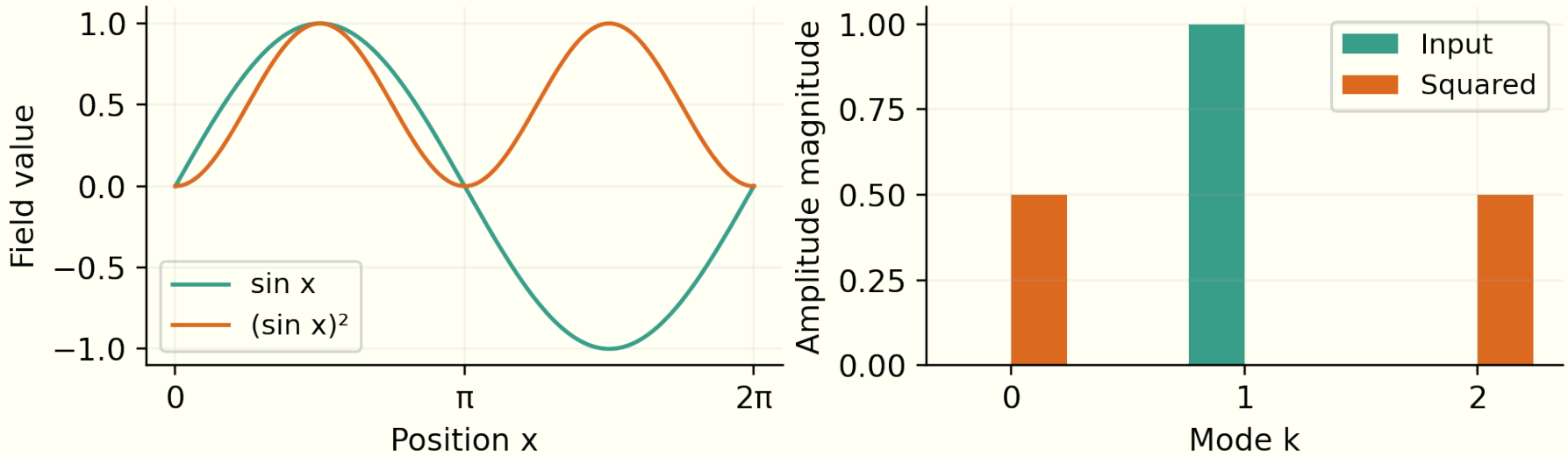
$$
\begin{aligned}
z(x) &= \mathcal{F}^{-1}[R_\ell \mathcal{F}(v_\ell)](x), \\
y(x) &= W_\ell v_\ell(x) + b_\ell, \\
v_{\ell+1}(x) &= \sigma(z(x) + y(x)).
\end{aligned}
$$

- The spectral path combines information across the domain.
- W_ℓ mixes channels at each position without a spectral cutoff.
- σ supplies nonlinear frequency interactions.

In multiple dimensions, k is a vector and the same channel formula applies.

A nonlinear operation creates new frequencies

$$\sin^2 x = \frac{1}{2} - \frac{1}{2}\cos 2x.$$



A multiplier at mode 2 gives $R(2) \times 0 = 0$ for the original input. A pointwise nonlinear operation can generate that missing mode.

Pseudospectral evaluation and an FNO block

Pseudospectral Burgers

1. Start with coefficients of u.
2. Apply known multipliers 1 and ik.
3. IFFT to values of u and u_x.
4. Form the product $u\,u_x$ on the grid.
5. FFT the product and integrate the PDE in time.

FNO features

1. Start with a feature field v_ℓ.
2. FFT and apply learned $R_\ell(k)$.
3. IFFT and add $W_\ell v_\ell + b_\ell$.
4. Apply a pointwise activation.
5. Continue to the next feature layer.

The nonlinear spatial operations couple modes. FNO layers need not represent physical time steps. McCabe et al., Section 2 and Appendix A.

Outline

1. From CNNs to Kernel Operators: The Conceptual Leap
2. Fourier Transform: The Mathematical Foundation
3. Fourier Neural Operator Architecture
4. Example 1: 1D Burgers Equation
5. Example 2: 2D Darcy Flow

The 1D Viscous Burgers Equation

Problem Formulation:
The 1D viscous Burgers equation models shock wave propagation:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad x \in [0, 2\pi], t \in [0, T]$$

with periodic boundary conditions and initial condition $u(x, 0) = u_0(x)$.

Operator learning task: Learn the mapping

$$\mathcal{G} : u_0(x) \mapsto u(x, T)$$

from initial condition to solution at time $T = 1$.

Dataset: 1024 training samples from varied initial conditions, solved using spectral methods.

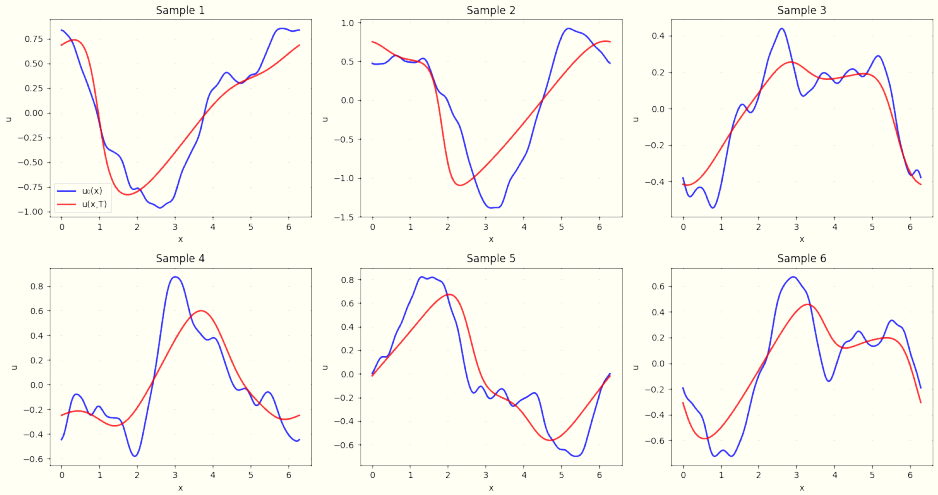
Burgers Equation: Physical Interpretation

The physics:

- **Nonlinear advection:** $u\frac{\partial u}{\partial x}$
- **Viscous diffusion:** $\nu\frac{\partial^2 u}{\partial x^2}$
- Competition creates shock waves
- Periodic boundary conditions

Why this problem:

- Prototype for nonlinear PDEs
- Captures shock formation
- Tests operator learning on complex dynamics



Sample initial conditions (blue) and evolved solutions (red) showing shock formation.

1D FNO Architecture and Training

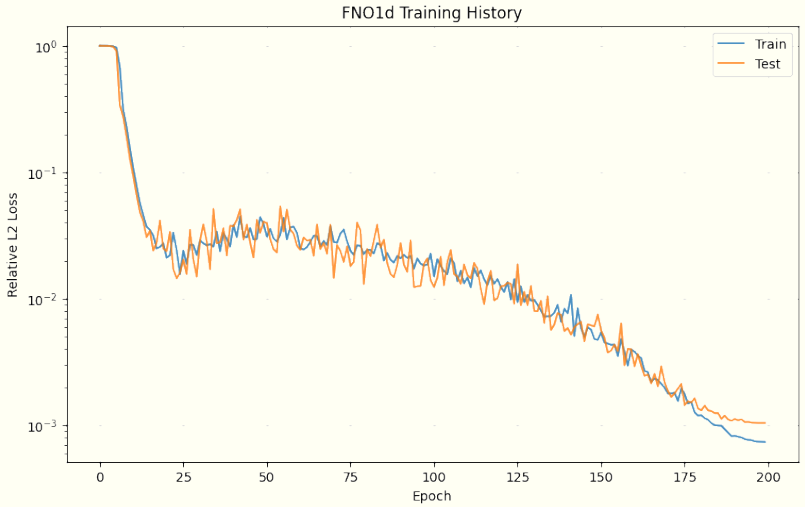
Model Configuration:

- **Input:** $(u_0(x), x)$ at $m = 2048$ points (subsampled to 512)
- **Fourier modes:** $k_{\max} = 16$
- **Hidden width:** $w = 64$
- **Layers:** 4 Fourier layers with skip connections
- **Activation:** GELU
- **Parameters:** $\sim$287K

Training Configuration:

- **Loss:** Relative L^2 loss: $\frac{\|u_{pred} - u_{true}\|_2}{\|u_{true}\|_2}$
- **Optimizer:** Adam with OneCycleLR scheduler
- **Training samples:** 1000
- **Test samples:** 100
- **Epochs:** 200

Burgers Equation: Training Results

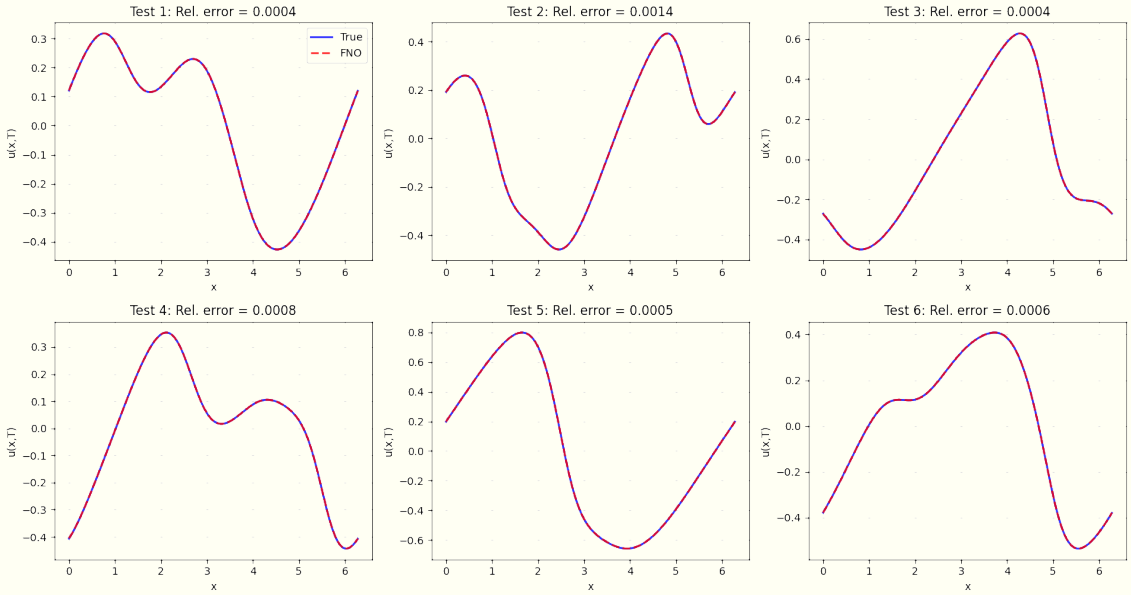


Training history showing convergence of relative L^2 loss.

Observations:

- Rapid convergence in first 50 epochs
- Final test error: $\sim$0.01-0.02 relative L^2
- No overfitting - train and test losses track together
- Model learns complex shock dynamics from data

Burgers Equation: Prediction Quality

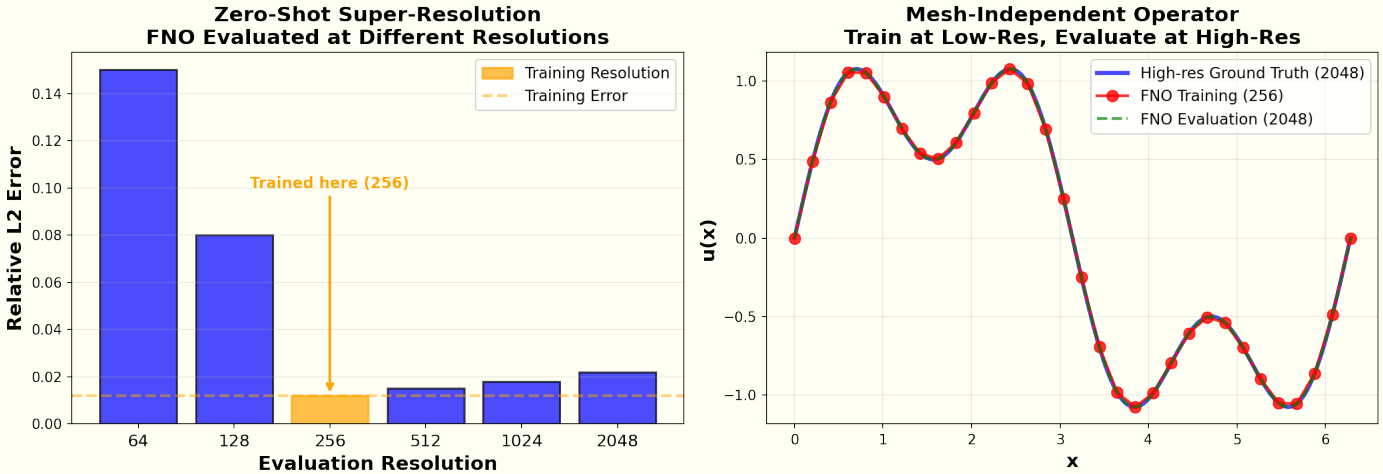


FNO predictions (red dashed) vs. true solutions (blue) for test cases.

Key results:

- Accurate prediction of shock locations and amplitudes
- Relative errors: 1-4%
- Works for initial conditions not seen during training
- Captures steep gradients despite spectral representation

Zero-Shot Super-Resolution



Training at 512 resolution, evaluating at 2048 resolution (4x upsampling).

Mesh Independence

- **Train:** 512 grid points
- **Test:** 2048 grid points (4x refinement)
- **Reported result:** Similar accuracy for this example

The same mode labels retain their meaning on the same physical domain. Shared parameters do not guarantee equal accuracy on every grid.

Outline

1. From CNNs to Kernel Operators: The Conceptual Leap
2. Fourier Transform: The Mathematical Foundation
3. Fourier Neural Operator Architecture
4. Example 1: 1D Burgers Equation
5. Example 2: 2D Darcy Flow

The 2D Darcy Flow Equation

Problem Formulation:
The 2D Darcy flow equation models steady-state flow in porous media:

$$-\nabla \cdot (a(x,y)\nabla u(x,y)) = f(x,y), \quad (x,y) \in [0,1]^2$$

with zero boundary conditions: $u|_{\partial\Omega} = 0$.

Operator learning task: Learn the mapping

$$\mathcal{G} : a(x,y) \mapsto u(x,y)$$

from permeability coefficient a to pressure/hydraulic head u.

Physical interpretation:

- $a(x,y)$: permeability field (how easily fluid flows)
- $u(x,y)$: pressure field
- $f(x,y)$: source term (set to 1)

A variable coefficient changes the operator

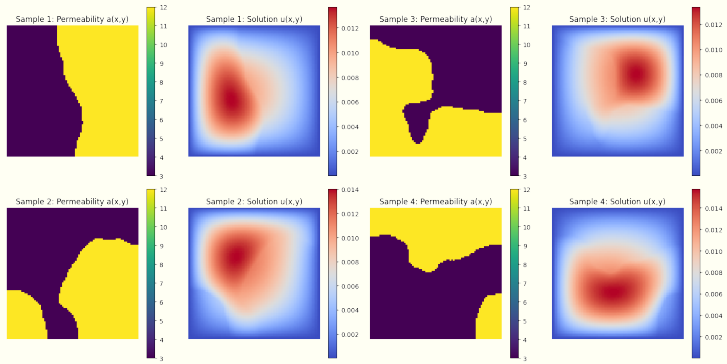
For each fixed permeability a, Darcy flow is linear in its forcing,

$$u(x) = \int_\Omega G_a(x, y) f(y)\, dy.$$

The Green's function depends on a and the boundary conditions. It generally cannot be written as one fixed $g(x - y)$.

- Multiplication by a spatially varying a couples Fourier modes.
- The learned task $a \mapsto u$ changes the PDE operator itself.
- Nonlinear feature layers and position information support this more general map.

Darcy Flow: Dataset and Physics



Left: Permeability fields $a(x, y)$ (input). Right: Pressure fields $u(x, y)$ (output).

Dataset characteristics:

- 1024 samples with random piecewise constant permeability
- Resolution: 241×241 (subsampled to 81×81)
- 1000 training samples, 100 test samples

Physical insight: Low permeability (dark) $\rightarrow$ high pressure gradients (red/yellow)

2D FNO Architecture

Model Configuration:

- **Input:** $(a(x, y), x, y)$ at 81×81 grid
- **Fourier modes:** $k_{\max} = 12$ (both x and y directions)
- **Hidden width:** $w = 32$
- **Layers:** 4 Fourier layers
- **Parameters:** $\sim$130K

Spectral Convolution 2D:

$$\text{FFT2} \rightarrow \text{Multiply modes } [:12, :12] \text{ and } [-12:, :12] \rightarrow \text{IFFT2}$$

Why two weight matrices? Due to symmetry of real FFT (rfft2), we need weights for both lower and upper frequencies.

Darcy Flow: Training Configuration

Training Setup:

- **Loss:** Relative L^2 loss in 2D
- **Optimizer:** Adam with step decay (StepLR)
- **Learning rate:** 0.001 with decay every 50 epochs
- **Batch size:** 20
- **Epochs:** 200 (500 in original paper)

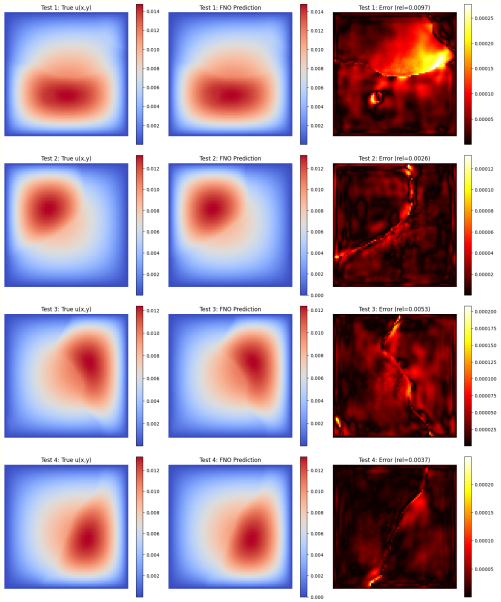
Data Normalization:

- Input permeability: Unit Gaussian normalization
- Output pressure: Unit Gaussian normalization
- Decode predictions before computing loss

Important

Normalize data for stable training, but always compute loss on physical quantities!

Darcy Flow: Results and Analysis



Left: True pressure. Middle: FNO prediction. Right: Error map.

Quantitative Results:

- Relative L^2 error: 2.9-3.2%
- Captures circular pressure distributions
- Handles heterogeneous permeability fields
- Structured errors (not random noise) suggest room for improvement

FNO properties

Shared mode weights

On the same physical domain, a retained wave number has the same meaning on different grids. Accuracy still depends on sampling, retained modes, and nonlinear aliasing.

Application cost

FFTs cost $O(N \log N)$ per fixed channel. Mode-wise channel transforms cost $O(MC_{\text{in}}C_{\text{out}})$. Training and reference-data generation add an upfront cost.

Global interaction

The learned convolution kernel can have global support even with a finite set of modes. Nonlinear blocks combine these global transformations with local operations.

Discretization, data, and physical constraints

Domain and grid

The standard FFT uses a regular grid and a periodic extension. Other boundaries and geometries need suitable representations.

Training coverage

Supervised learning needs reference input-output pairs. Unseen coefficients, forcing functions, or fine scales require evaluation.

Physical constraints

A standard prediction loss does not enforce the PDE or conservation laws. PDE-aware losses and explicit constraints can address particular requirements.

FNO vs DeepONet: Comparison

Aspect	DeepONet	FNO
Architecture	Branch-Trunk	Spectral Convolution
Space	Physical	Fourier
Queries	Arbitrary points	Grid points (FFT)
Input handling	Branch encoding	FFT on a uniform grid
Parameters	Branch/trunk dependent	Width/mode dependent
Global interaction	Learned basis coefficients	Spectral convolution
Output sampling	Coordinate queries	FFT grid in this model

Key difference:

- **DeepONet:** Learns basis decomposition in physical space
- **FNO:** Learns multiplication in Fourier space

The choice depends on input sampling, domain representation, and required output queries.

- Encode the boundary conditions and geometry explicitly
- Match the query pattern to the model implementation

Extensions to FNO

Geo-FNO: Arbitrary Geometries

- Use coordinate transforms to handle non-periodic domains
- Applies Fourier layers in transformed space
- Enables FNO for complex geometries

Physics-Informed FNO

- Add PDE residual to loss function
- Penalizes PDE residuals on the evaluated samples
- Reduces data requirements

Factorized FNO

- Low-rank approximations for 3D problems
- Reduces memory and computation
- Enables high-resolution 3D operator learning

Implementation Tips

1. Architecture Choices

- **Fourier modes:** Start with 12-16, adjust based on problem
- **Width:** 32-64 for most problems
- **Layers:** 4 is standard, more for complex operators
- **Activation:** GELU works well (smoother than ReLU)

2. Training Strategy

- **Loss:** Relative L^2 for scale-invariance
- **Optimizer:** Adam with learning rate scheduling
- **Normalization:** Unit Gaussian for inputs and outputs
- **Epochs:** 200-500 depending on problem complexity

3. Validation

- Test on different resolutions (super-resolution check)

Resources and Further Reading

Original Paper:

- Li et al., "Fourier Neural Operator for Parametric PDEs" (ICLR 2021)
- `https://arxiv.org/abs/2010.08895`

Extensions:

- Geo-FNO: `https://arxiv.org/abs/2207.05209`
- Physics-Informed Neural Operators: `https://arxiv.org/abs/2111.03794`
- Neural Operator Review: `https://arxiv.org/abs/2108.08481`

Code and Data:

- Official implementation: `https://github.com/neuraloperator/neuraloperator`
- Course notebooks: `https://github.com/kks32-courses/sciml`
- Dataset: `https://huggingface.co/datasets/kks32/sciml-dataset`

Questions?

Thank you!

Contact:

Krishna Kumar

krishnak@utexas.edu

University of Texas at Austin

Interactive Demo:

▸ FNO Notebook

Dataset:

▸ Hugging Face Dataset